



Dependency Injection

Persistent Interactive | Persistent University



Agenda

- Need for Dependency Injection
- Implementation in Angular
- Need for Providers
- Providing in module
- Providing in component

What is Dependency Injection

- Dependency Injection is a software design in which components are given their dependencies instead of hard coding them within the component.
- This responsibility of creating concrete instances of dependencies is moved out of the components and instead allows dependencies to be configurable.
- It also helps in making components reusable, maintainable and testable.
- Dependency injection is an important application design pattern it's used not only in Angular but throughout software development.
- Angular has its own dependency injection framework, and we really can't build an Angular application without it.
- It's used so widely that almost everyone just calls it **DI**.

Example of Car class

1. Difficult to maintain the code over time since instances are being created in the Car.
2. Eg: If Engine constructor changes in future to accept a parameter, then this code will break.
3. Changing the door type involves code change in the Car.
4. Difficult to unit test the classes.
5. Data sharing is not possible

```
class Car {  
    constructor() {  
        this.engine = new Engine();  
        this.tires = new Tires();  
        this.doors = new TypeADoors();  
    }  
}
```

Dependencies of the Car

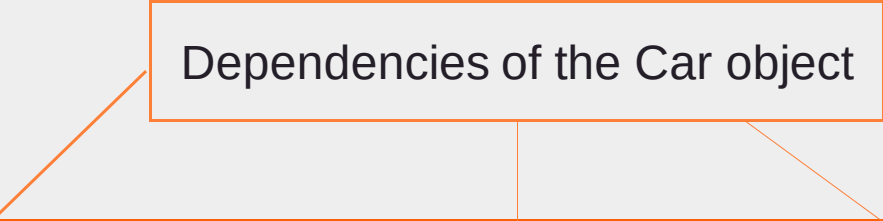
Take the
instances as parameters

```
class Car {  
    constructor(@Inject(Engine),@Inject (Tires),@Inject(Doors))  
    {  
        this.engine = engine;  
        this.tires = tires;  
        this.doors = doors;  
    }  
}
```

Car Creator

- The creator of the Car instance will create and pass the instances of engine, tires and doors.

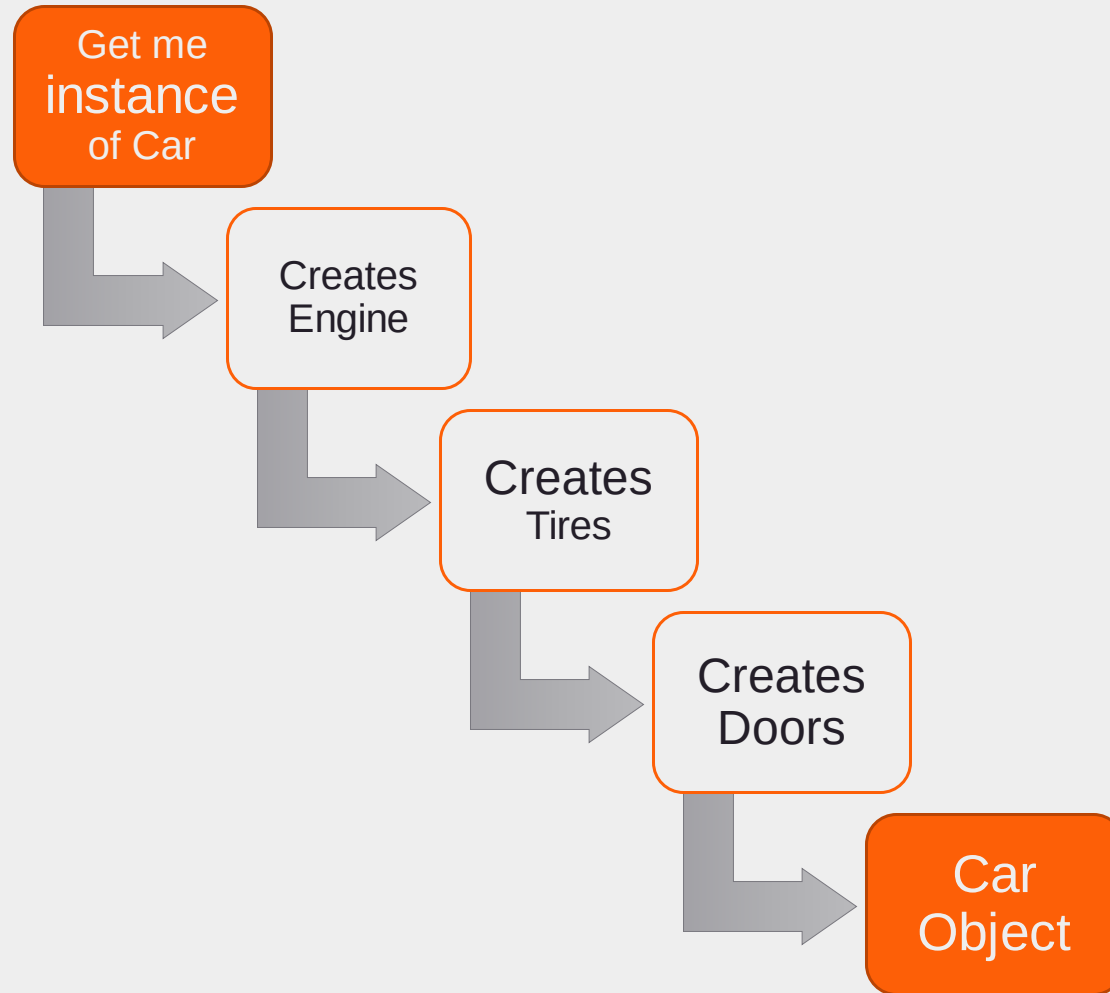
Dependencies of the Car object



```
var car = new Car(new Engine(), new Tires(), new Doors());
```

- The Injector will create all these instances for the Car.

Dependency Injection Framework



Injector Class that gives Car object

- Angular has an inbuilt injector which handles all this.
- Injector Object will create an instance of Car.

```
let carObj = injector.get(Car);
```

- The Injector will create instance of Car along with dependencies.
- **How will it know to create dependencies? Which concrete classes to use for creating the instances.**

Defining Providers

- At the core of the DI framework is an *injector*.
- An injector is passed a *token* and returns an instance/reference of a *dependency*.
- **This is termed as Resolving a dependency.**

```
let injector = ReflectiveInjector.resolveAndCreate([  
  {provide:Car, useClass:Car},  
  {provide:Engine, useClass:Engine},  
  {provide:Tires, useClass:Tires},  
  {provide:Doors, useClass:TypeBDoors}  
]);  
let carObj = injector.get(Car);
```

Terminologies in DI

- **Token**
 - This uniquely identifies something that we want injected. A dependency of our code.
- **Dependency**
 - The actual code we want injected.
- **Provider**
 - This is a map between a token and a list of dependencies.
- **Injector**
 - This is a function which when passed a token returns a dependency (or a list of dependencies)

Example

```
class ProductsComponent {  
  dataService: DataService;  
  
  constructor() {  
    this. dataService = new AmazonDataService();  
  }  
  getProducts() {  
    this. dataService.getData();  
  }  
}
```

Better version of the code

- Pass the instance of product service in the constructor.

```
class ProductsComponent {  
  dataService: DataService;  
  constructor(dataservice:ProductService)  {  
    this. dataService = dataservice;  
  }  
  getProducts() {  
    this. dataService.getData();  
  }  
}
```

- Creation code is moved outside the class.

```
productcomp = new ProductComponent(new AmazonDataService());  
productcomp.getProducts();
```

Advantages of this approach

1. Flexibility to change the product service.

```
productcomp = new ProductComponent(new BigBasketDataService());  
productcomp.getProducts();
```

2. Easier to test since this approach allows to pass Mock services.

```
let mockService = MockedProductService();  
productcomp = new ProductComponent(mockService);
```

3. Easier to maintain. If the constructor of BigBasketDataService is changed, then the ProductComponent class is not affected.

Resolving the dependencies

```
class AmazonDataService {};  
class BigBasketDataService {};
```

- **Providers:** An object consisting of token and corresponding class name.
- A provider is like a recipe that tells the injector **how** to create an instance of a dependency.

```
let injector = ReflectiveInjector.resolveAndCreate([  
  { provide:"BigBasketDataService", useClass: BigBasketDataService },  
]);
```

Best place to invoke service methods?

- Services are always injected in the constructor.
- However, **ngOnInit()** is the best place to invoke the service methods.
- ngOnInit is life cycle hook to indicate that that Angular is done creating the component.
- Executed after the constructor.

Constructor vs ngOnInit

Constructor

- The Constructor is a default method of the class that is executed when the class is instantiated.
- Use only for initialization of class fields and injected services.
- Invoked when new() is executed.
- Eg: Default values for class members.

ngOnInit

- ngOnInit is life cycle hook to indicate that that Angular is done creating the component.
- Any other initialization or business logic to be applied on the class data is to be executed here.
- Executed after the constructor.
- Eg: Fetch data from server and convert the format.

Switching Dependencies

- Suppose we decided to change the email service from Mandrill to SendGrid, then what changes will be required?
- We would have to make changes at all occurrences of Mandrill and replace with SendGrid.
- Instead, we can use DI to switch the dependency like below.

```
let injector = ReflectiveInjector.resolveAndCreate([  
  { provide: "EmailService", useClass: MandrillService }  
]);
```

```
let emailService = injector.get("EmailService");  
console.log(emailService); // new MandrillService()
```

Same token in multiple provides

- When configuring an injector with the *same* token multiple times, the last provider just *overwrites* the previous providers.

```
let injector = ReflectiveInjector.resolveAndCreate([  
  { provide: "EmailService", useClass: SendGridService },  
  { provide: "EmailService", useClass: MandrillService },  
]);
```

Shorthand way to define Providers

- As long as they adhere to same interface, the below two statements are equivalent.

```
Providers: [{provide:'SendGridService',useClass:'SendGridService'}]
```

```
Providers:[SendGridService]
```

Dependency Caching

```
@Component({
  selector: 'app-products',
  templateUrl: './products.component.html',
  providers:[{provide:
AmazonDataService,useClass:AmazonDataService}]
})
export class ProductsComponent{
  constructor(private pservice:AmazonDataService) {
  }
}
```

```
Let serviceObj1 = injector.get(AmazonDataService);
Let serviceObj2 = injector.get(AmazonDataService);
```

- **Will the two objects be same?**

Dependency Caching

- The dependencies returned from injectors are cached.
- So multiple calls to the same injector for the same token will return the same instance.

Single instance of Dependencies

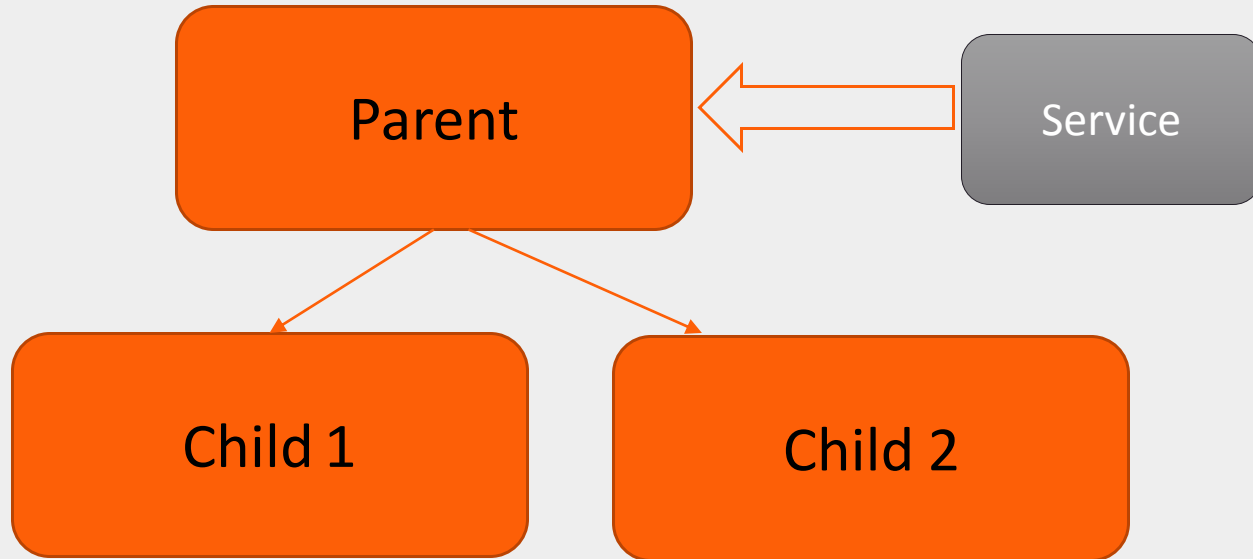
- Not that multiple calls to the *same* injector for the *same* token will return the *same* instance.

```
let emailService1 = injector.get(MandrillService);  
let emailService2 = injector.get(MandrillService);  
console.log(emailService1 === emailService2); //returns true
```

- A **different injector** for the *same* token might **return a different instance** of a dependency but the *same* injector will always return the *same* instance.
- We can share *state* between two different parts of our application by injecting the same dependency,

Sharing a Service instance

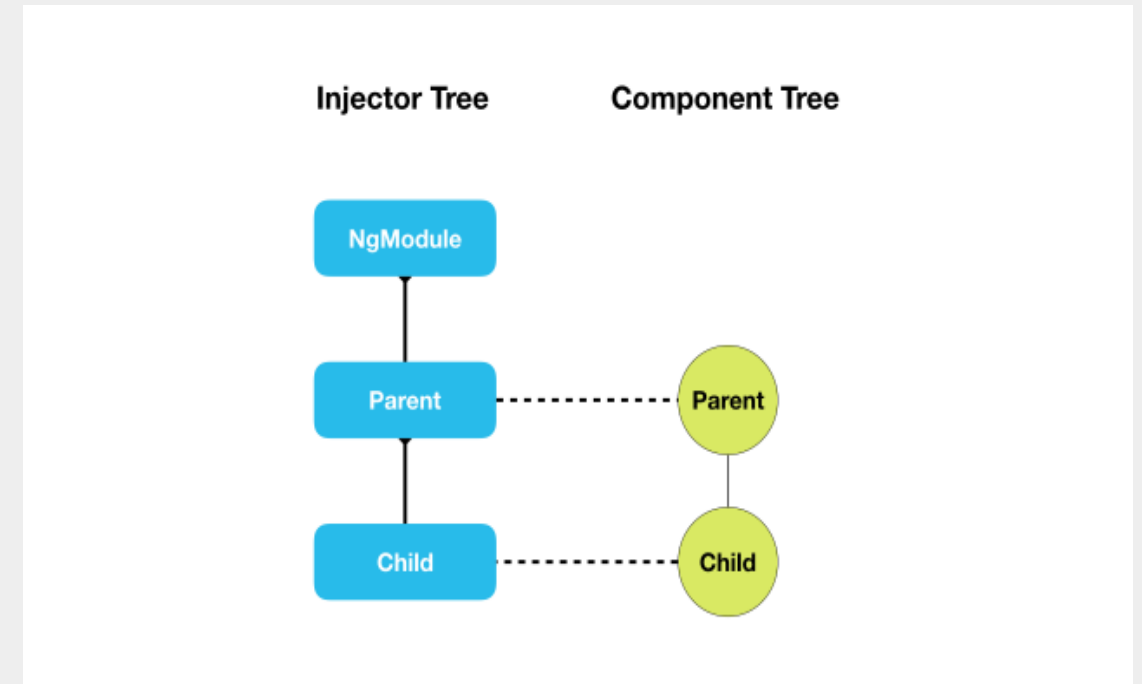
- Dependencies are singletons within the scope of an injector.



- If the service is provided in Parent, then all the 3 classes get the same instance of the service.

Injector Tree

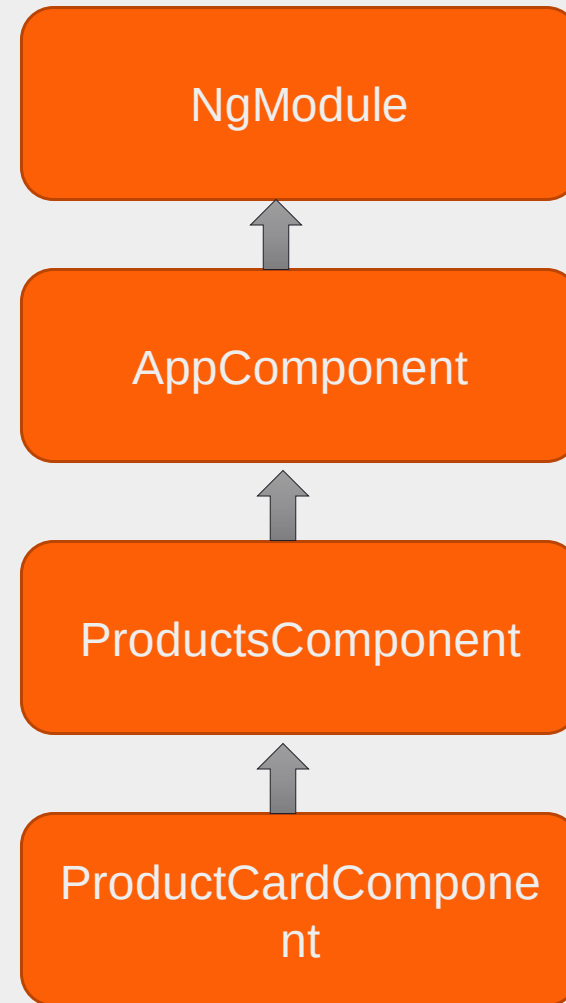
- Injector tree resembles the Component tree.
- Top level parent Injector at @NgModule
- The subsequent child component **Parent** will have child injector that stems from NgModule.
- The **Child** component of Parent will have child injector stemming from Parent.



Ref: <https://codecraft.tv/>

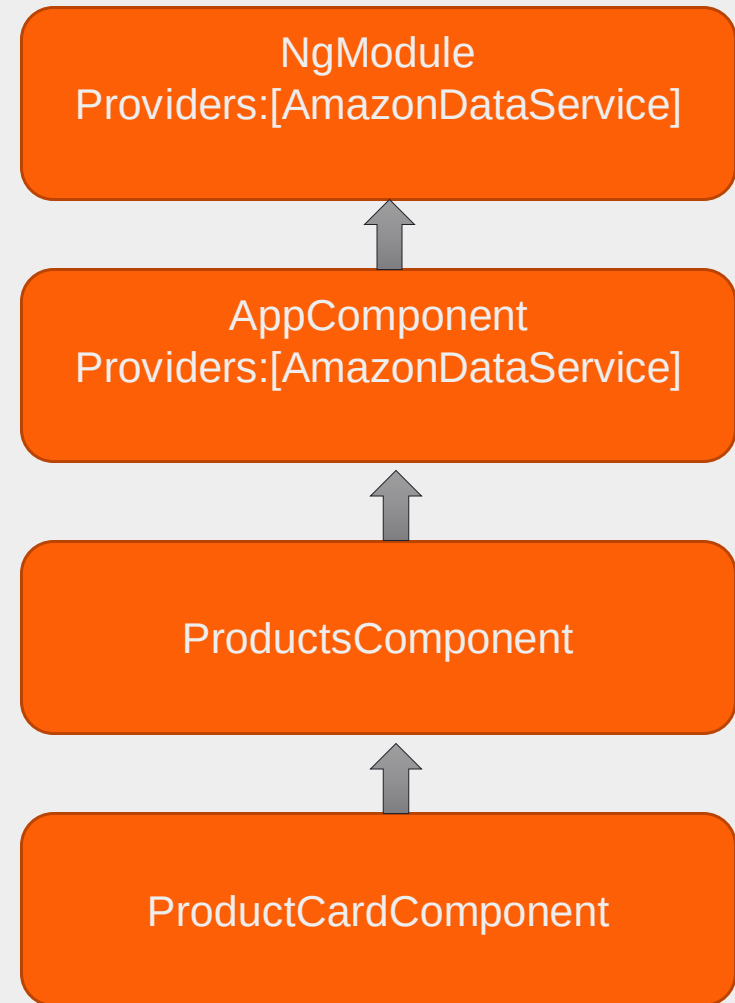
Resolving dependencies

- Providers can be configured at any level using `Providers[]`.
- If providers not configured in a component's injector then Angular checks its immediate Parent if it has been configured.
- If not found, then the process repeats for the next level parent until it finds one.
- The last check is done at NgModule level. If still not found then an error is thrown saying “**No Provider found**”.

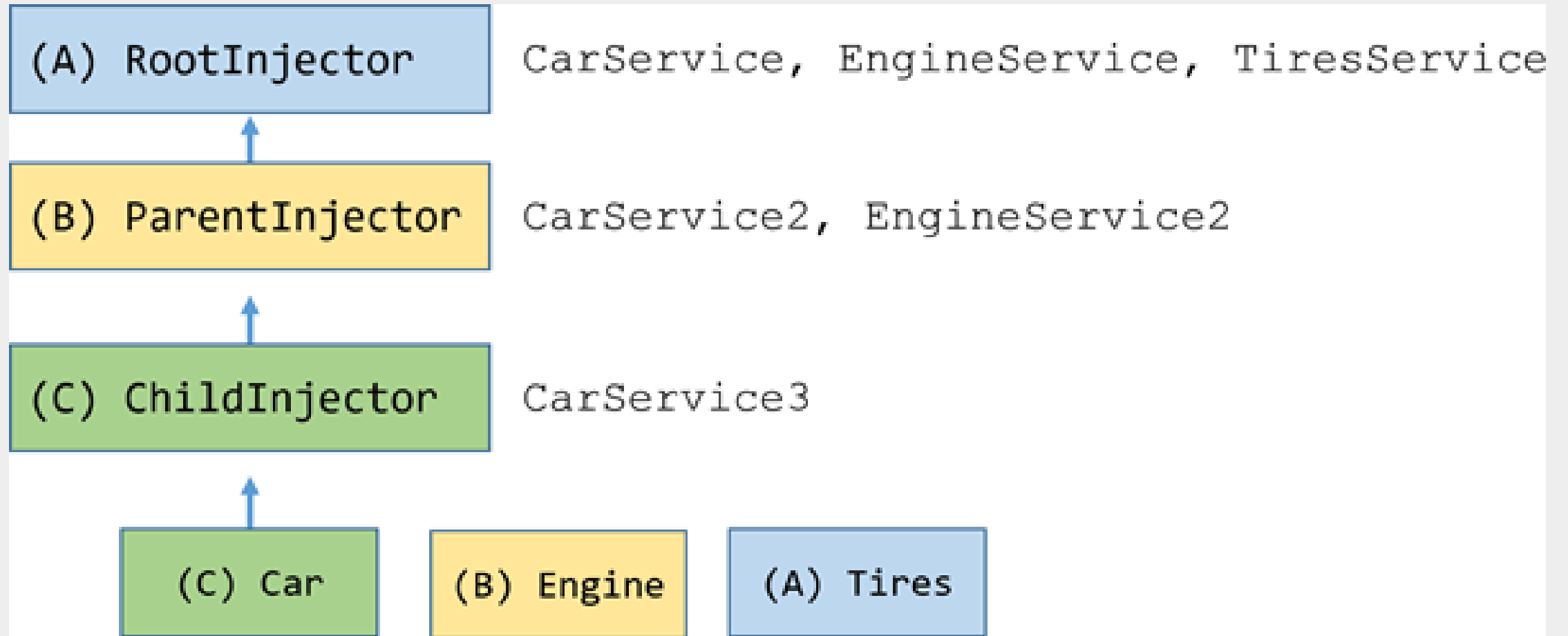


Resolving dependencies

```
export class ProductCardComponent{  
  constructor(dataserv:AmazonDataService)  
  {  
    //How will the dependency get resolved?  
  }  
}
```



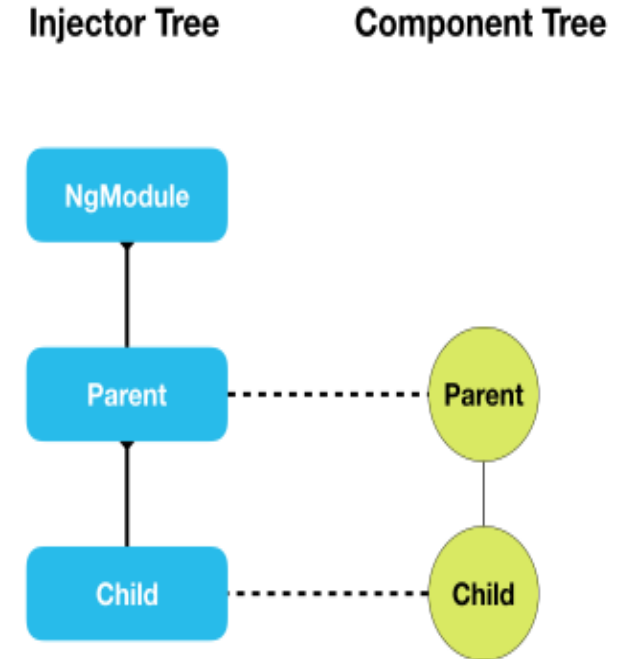
How dependencies are resolved?



Ref: <https://codecraft.tv/>

The Injector Tree

- An Angular application has a tree of injectors that resembles the component tree.
- **Parent Injector**
 - NgModule
- **Child Injectors**
 - Parent
 - Child



Ref: <https://codecraft.tv/>

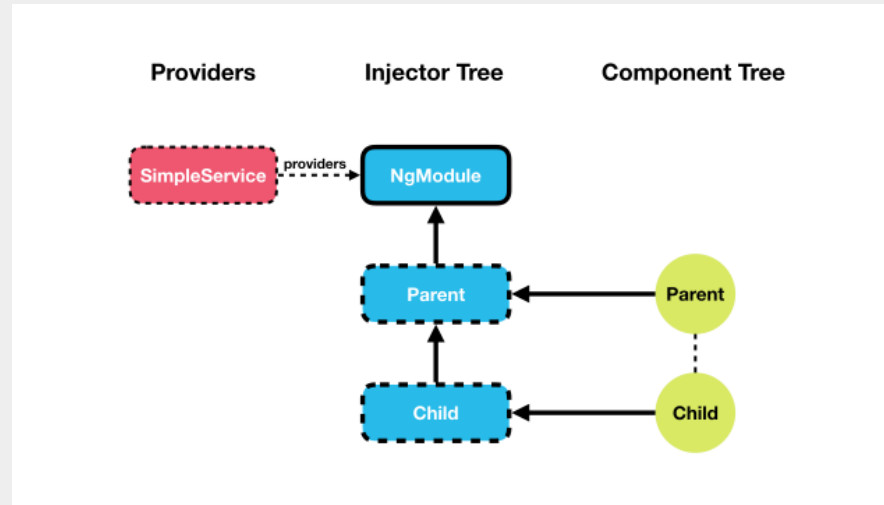
Providers in the module

- @NgModule has a providers array where dependencies can be configured.
- The **parent injector** is present at this level.

```
@NgModule({  
  imports:  
  declarations:  
  bootstrap:  
  providers:[AmazonDataService]  
})
```

Injector at NgModule

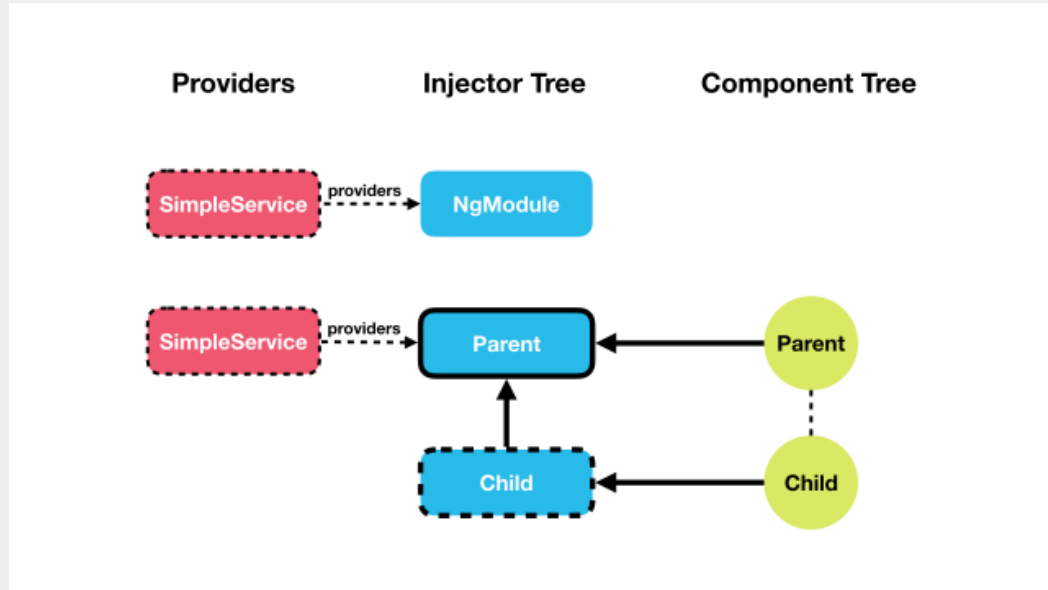
- Single Injector is configured here at NgModule to resolve dependencies.
- Every-time we request an instance of SimpleService to be injected into one of our components it's always going to inject the same instance.



Ref: <https://codecraft.tv/>

- This configuration can be done when ALL the components of the module need to share the **same instance of the service**.

Injector at Component level

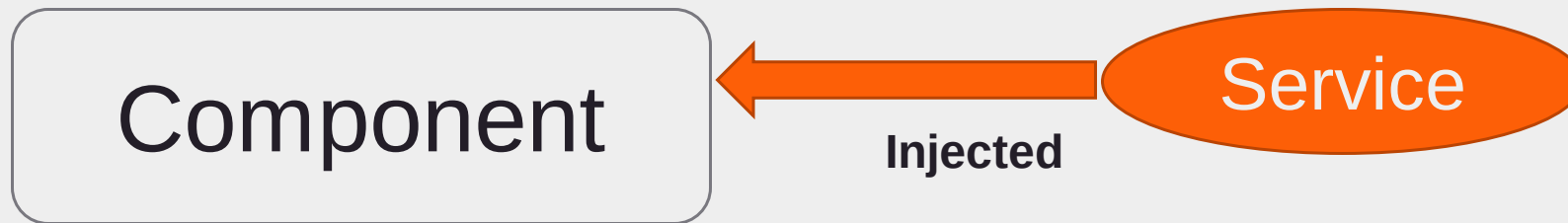


Ref: <https://codecraft.tv/>

- Each instance of `ParentComponent` now has its own instance of `SimpleService`, so state is not shared globally but only between a `ParentComponent` and its child components.
- If we want to have one instance of a service per component, and shared with all the components children, we configure it on the `providers` property on our component decorator.

Define Services

- Services contain common code which needs to be shared across multiple components or entire application.
 - List of Users, Products, Configurations
 - Functionality like authentication
 - RESTful calls to fetch data.
- Components use services. Hence Services are injected into Components



Injecting a Service into a Service

- A service can in turn have a dependency on another service like below:

```
class AmazonDataService {  
    constructor(private service: SimpleService) {  
  
};  
}
```

```
class SimpleService {  
  
}
```

- This results in an error like

Can't resolve all parameters for AmazonDataService: (?).

@Inject()

- The below will resolve the error.

```
class AmazonDataService {  
    constructor(@Inject(SimpleService) private simpleservice:SimpleService) {  
  
    }  
}
```

- Remember to add a provider to SimpleService at the appropriate component or Module level.

@Injectable()

- Decorating all constructor arguments with @Inject can be tiresome.
- Instead we can decorate the entire class with @Injectable.

```
@Injectable()  
class AmazonDataService {  
    constructor( private simpleservice:SimpleService) {  
    }  
}
```

- @Injectable is actually a shortcut for having to decorate every parameter in the constructor with @Inject.

@Injectable for Components

- The other decorators in Angular, such as @Component and @Directive, already perform the same function as @Injectable.
- Hence we only need to use @Injectable on classes which don't already use one of the other Angular decorators.

Dynamically Injecting Services

```
@Component({  
  providers: [FBService, TWService,GService]  
})  
export class LoginComponent {  
  constructor(private injector: Injector) { }  
  init(){  
    //Based on User Input, set the service  
    authService = injector.get(FBService);  
  }  
}
```

Passing parameters to Services

- Create a file app.config.ts

```
import { OpaqueToken } from "@angular/core";

export let APP_CONFIG = new
OpaqueToken("app.config");

export interface IAppConfig {
  apiEndpoint: string;
}

export const AppConfig: IAppConfig = {
  apiEndpoint: "http://localhost:15422/api/"
};
```

Passing parameters to Services

- In app.module.ts

```
import { APP_CONFIG, AppConfig } from './app.config';

@NgModule({
  providers: [
    { provide: APP_CONFIG, useValue: AppConfig }
  ]
})
```

Passing parameters to Services

- In <name>.service.ts

```
import { APP_CONFIG, IAppConfig } from './app.config';

@Injectable()
export class YourService {

    constructor(@Inject(APP_CONFIG) private config: IAppConfig) {
        // You can use config.apiEndpoint now
    }
}
```

When type not specified in DI

- Notice below, the type for otherSimple is not specified in the constructor.

```
@Injectable()
class SimpleService {
  otherSimple: OtherSimpleService;

  constructor(otherSimple: any) {
    this.otherSimple = otherSimple;
  };
}
```

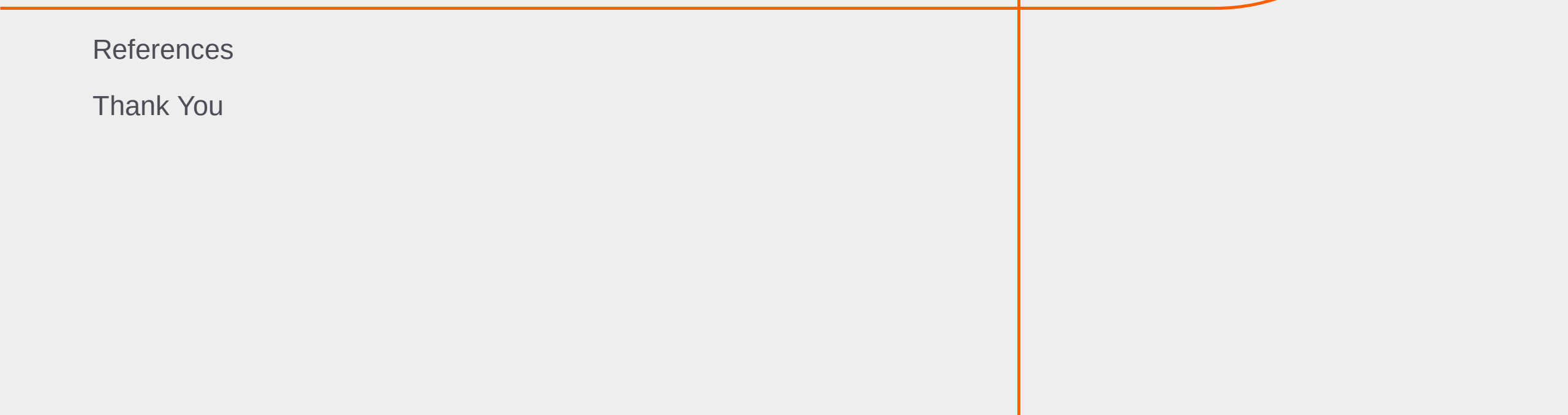
- The DI framework doesn't know what to inject and therefore **Can't resolve all parameters for SimpleService: (?)**. is printed to the console.

Summary: Session

With this we have come to an end of our session, where we discussed about

- What are dependencies?
- Why Dependency Injection?
- Implementation in Angular
- Importance of Providers
- What is Root Provider

Appendix



References

Thank You

Reference Links

- <http://blog.thoughttram.io/angular/2015/05/18/dependency-injection-in-angular-2.html>
- <http://blog.thoughttram.io/angular/2015/09/17/resolve-service-dependencies-in-angular-2.html>
- <https://angular.io/docs/ts/latest/guide/dependency-injection.html>

Key Contacts :

Persistent University :

- Archana Ghatigar
archana_ghatigar@persistent.co.in



Thank you!

Persistent Interactive | Persistent University

